

Data Science

Preprocessing

Prof. Rayson Laroca
rayson@ppgia.pucpr.br

Learning Goals

- Understand why raw features can harm model performance
- Encode categorical variables appropriately
- Understand common strategies for transforming numerical features
 - MinMaxScaler, StandardScaler, RobustScaler, and Box-Cox
- Handle missing data without introducing bias
- Avoid data leakage during preprocessing

Why Preprocessing Matters

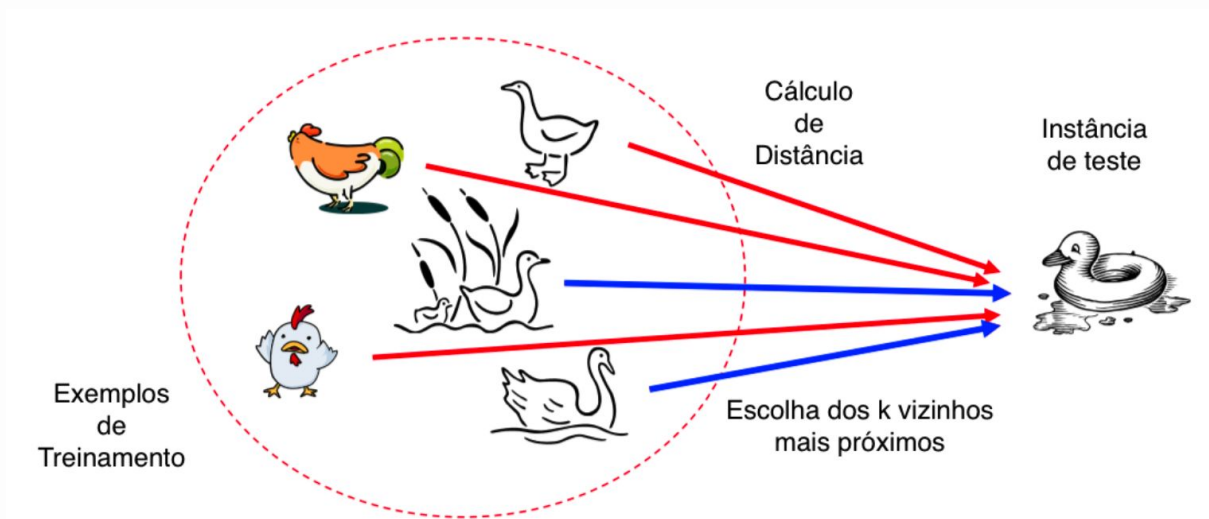
- ML models do not “understand” raw data as humans do
 - Feature scale, type, distribution, and missingness affect learning
- Good preprocessing improves stability, convergence, and generalization
- Algorithms Sensitive to Scale
 - K-Nearest Neighbors (KNN), Logistic and linear regression, Neural networks, SVM with RBF kernel, among others

KNN as a Motivating Example

- KNN is a lazy learning algorithm
 - Training: stores the training examples
 - Testing: compares a new instance with stored examples
 - The predicted label depends on the nearest neighbors

KNN: Core Assumption

- **Similar** examples should have similar labels
 - If it looks like a duck, swims like a duck, and quacks like a duck, then it probably *is* a duck



KNN: Core Assumption

- **Similar** examples should have similar labels
 - Similarity is usually measured through distance
 - Feature representation directly affects what “similar” means

Distance-Based Learning

- Each feature becomes a dimension
- Each instance becomes a point in feature space
- Euclidean distance is commonly used to compare instances

$$d(\vec{x}_1, \vec{x}_2) = \sqrt{\sum_{i=1}^d (\vec{x}_1[i] - \vec{x}_2[i])^2}$$

The Scaling Problem

- Problem with actual instances:

Idade	Renda	Nível de Criminalidade	Cidade	Valor
30	5.000	Baixo	Curitiba	200.000
60	6.000	Médio	Curitiba	50.000

- Idade: $(30-60)^2=900$
- Renda: $(5.000-6.000)^2=1.000.000$
- Nível de criminalidade: $(\text{Baixo} - \text{Médio}) = (1)^2$
- Cidade: $(0)^2$
- Valor: $(200.000-50.000)^2=225*10^8$
- $\text{sum}=900+1.000.000+1+0+225*10^8$

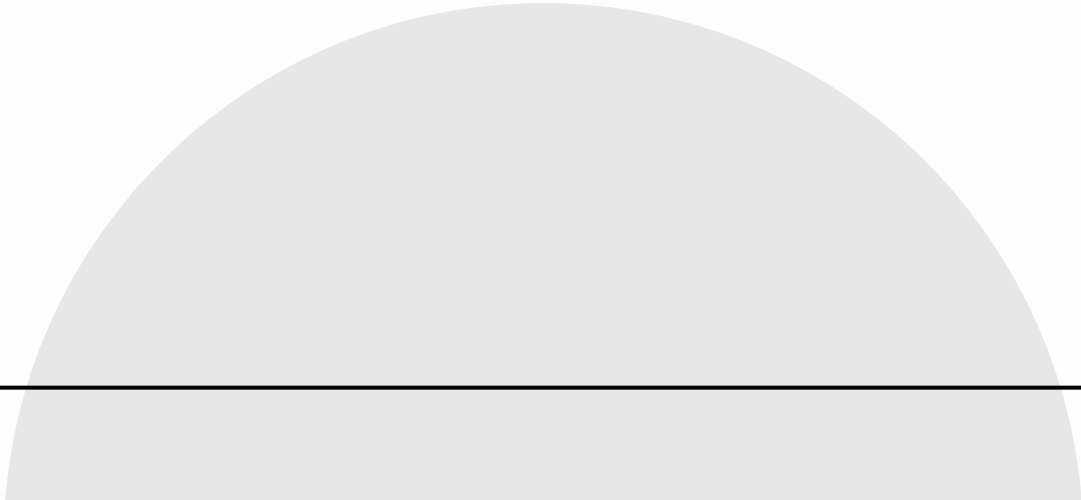
Without scaling, house value dominates the distance!

Distance = $\text{sqrt}(\text{sum}) = 150.003$

Key Takeaway from KNN

- Without scaling, KNN may become a “largest-scale-feature classifier”
 - The model may ignore important features with smaller ranges
- Scaling makes features more comparable
- The same issue can affect models based on distances, gradients, margins, or variance

CATEGORICAL VARIABLE REPRESENTATION



Categorical Variable Representation

- Many ML models require numerical inputs
- Categorical variables must be encoded carefully
- Poor encodings can introduce false relationships
 - Example?

Categorical Variable Representation

- Poor encodings can introduce false relationships. For example:
 - Variable: **City**
 - Original categories: **Curitiba, São Paulo, Rio de Janeiro**
 - Poor encoding:
 - Curitiba = 0
 - São Paulo = 1
 - Rio de Janeiro = 2
 - This encoding may accidentally suggest that Rio de Janeiro is “greater than” São Paulo, or that Curitiba is closer to São Paulo than to Rio de Janeiro.

Categorical Variables and Machine Learning

- Some algorithms are inherently capable of handling categorical variables, including:
 - Decision trees
 - Random Forests
 - Naïve Bayes
- However, libraries such as scikit-learn often require categorical variables to be converted into appropriate numerical representations before model training.

Types of Categorical Variables (**Recap**)

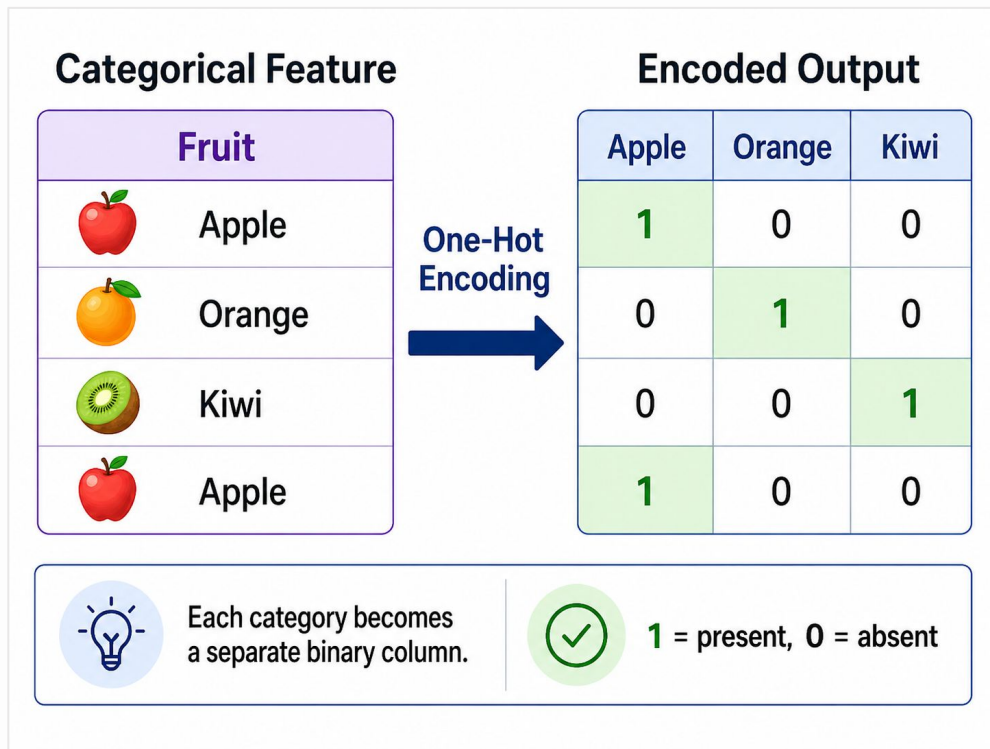
- Nominal (no natural order)
 - Colors (red, blue, green)
 - Gender (male, female)
- Ordinal (meaningful order)
 - Educational attainment (e.g., Elementary school, High school, Bachelor's, Master's, PhD)
 - Satisfaction ratings (poor, average, good, excellent)

Choosing an Encoding

- Use **one-hot encoding** for nominal variables
- Use **ordinal encoding** only when the order is meaningful
- Avoid creating artificial order where none exists

One-Hot Encoding

- Converts each category into a binary column
- Best for nominal variables
- Works well when the number of categories is small or moderate

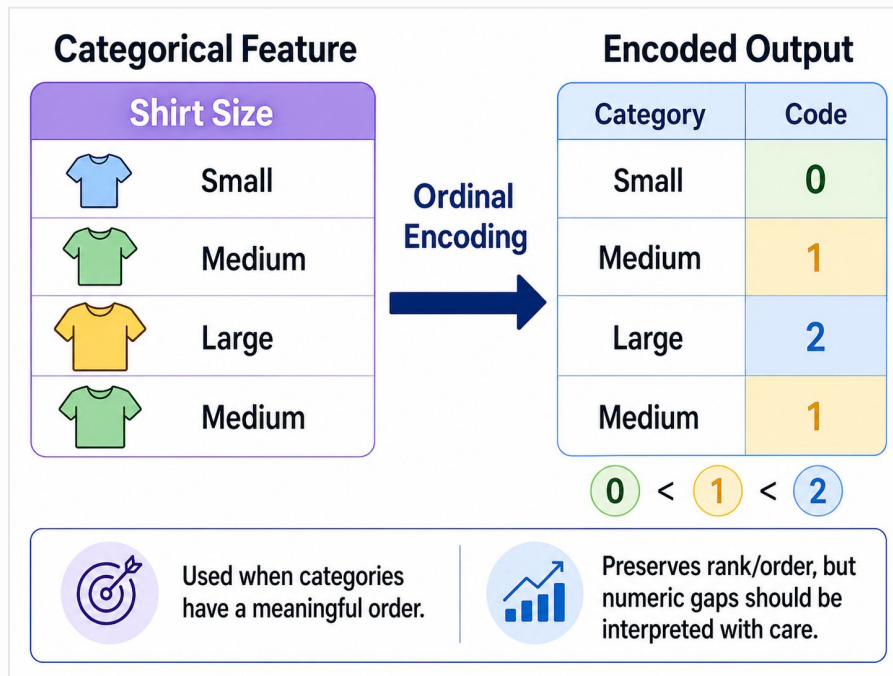


Limitations of One-Hot Encoding

- Can create many columns for high-cardinality variables
- May increase memory usage and model complexity
- Rare categories can produce sparse columns with very few positive examples
 - Models may struggle to learn reliable patterns from these rare categories
 - A common solution is to group rare categories into a broader category such as **“Other”**

Ordinal Encoding

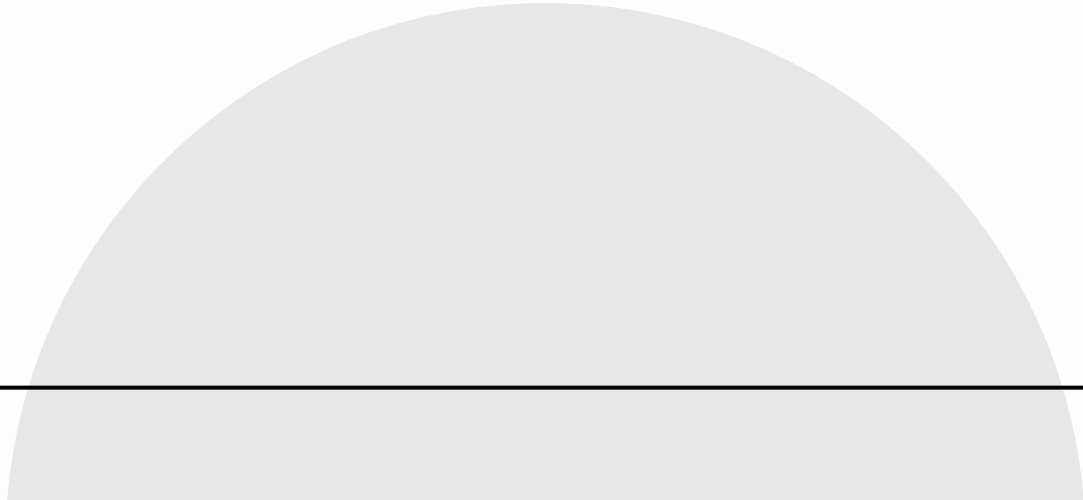
- Maps categories to ordered numerical values
- Suitable for variables with meaningful rank
- Compact and easy to interpret



Limitations of Ordinal Encoding

- Preserves order, but may introduce artificial distances
 - Models may assume that consecutive values are equally spaced
 - Example: High School = 0, Bachelor's = 1, Master's = 2, PhD = 3
 - The order is meaningful, but the distance between levels may not be uniform

NUMERICAL FEATURE TRANSFORMATIONS



Scaling, Standardization, and Normalization

- **Scaling:** changes the range of the values, usually to $[0, 1]$, while preserving the overall shape of the distribution
- **Standardization:** subtracts the mean and divides by the standard deviation, giving the feature **mean 0** and **standard deviation 1**
- **Normalization:** is often used ambiguously in practice. Some authors use it as a synonym for scaling, while others use it for different transformations. Always name the exact method applied

Practice (1/2)

- **MinMaxScaler**

- Rescales values to a fixed interval, usually $[0, 1]$
- Preserves the original distribution shape
- Sensitive to outliers

- **StandardScaler**

- Subtracts the mean and divides by the standard deviation
- Produces features with mean 0 and standard deviation 1
- $z = (x - \text{average}) / \text{standard deviation}$

Practice (2/2)

- **RobustScaler**

- Uses the median and interquartile range
- Less affected by outliers
- $z = (x - Q2) / (Q3 - Q1)$

- **Box-Cox (PowerTransformer, method='box-cox')**

- Attempts to make data more Gaussian-like
- Works only with strictly positive values
- Useful for highly skewed numerical variables

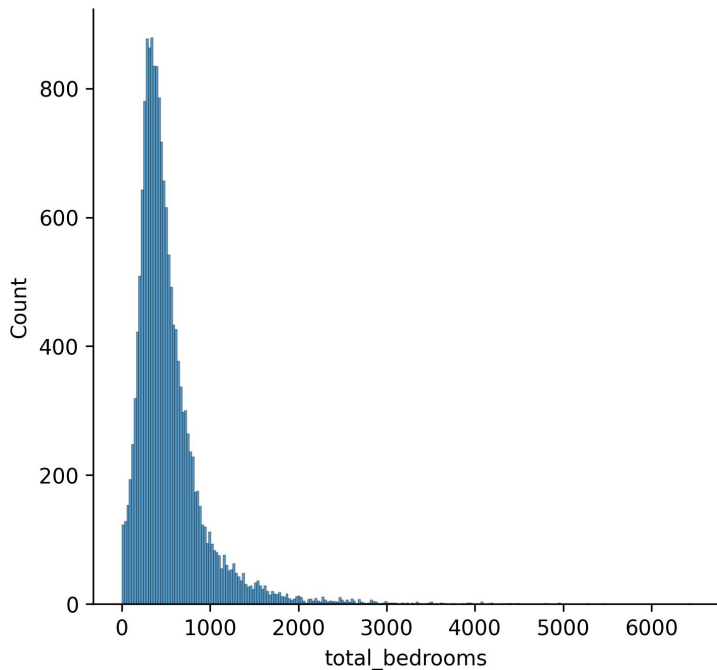
Comparing the Transformations

- **MinMaxScaler:** fixed range, outlier-sensitive
- **StandardScaler:** mean-centered, common default
- **RobustScaler:** better with outliers
- **Box-Cox:** changes distribution shape

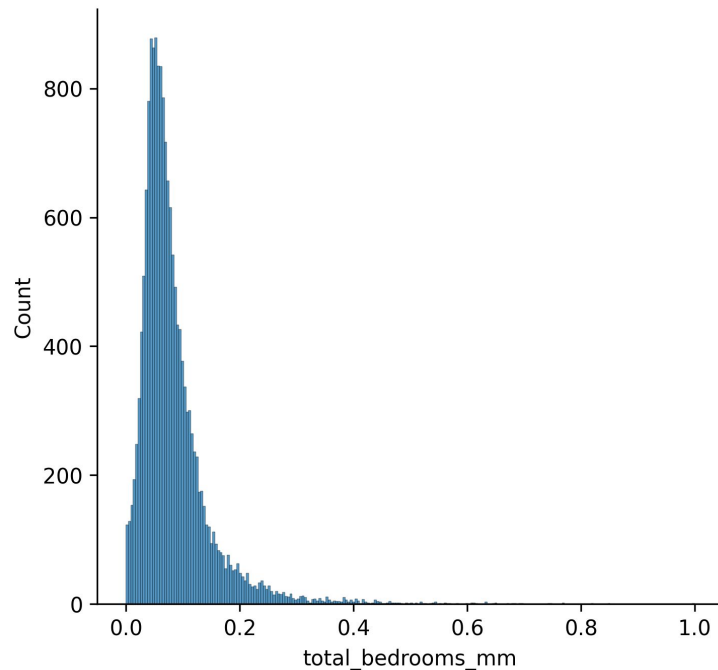
MinMaxScaler

```
mm = MinMaxScaler()  
df['total_bedrooms_mm'] =  
mm.fit_transform(np.array(df['total_bedrooms']).reshape(-1, 1))
```

Original



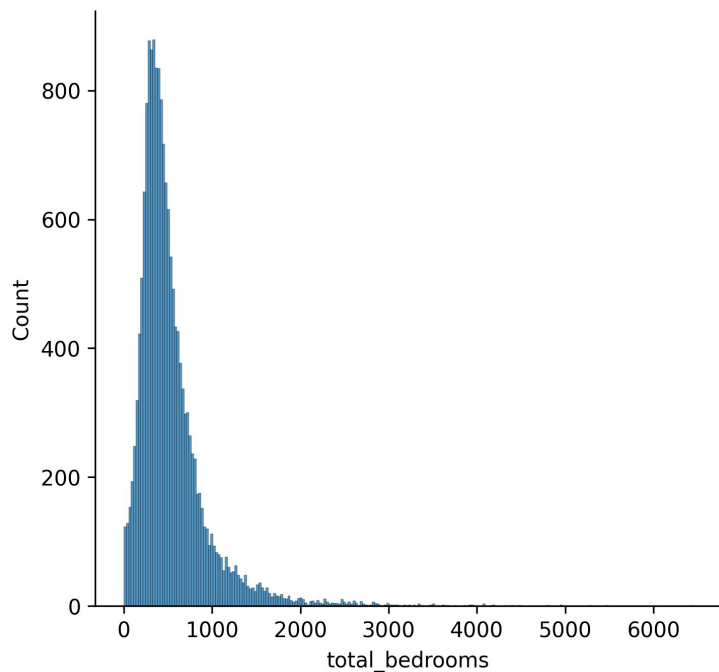
MinMaxScaler



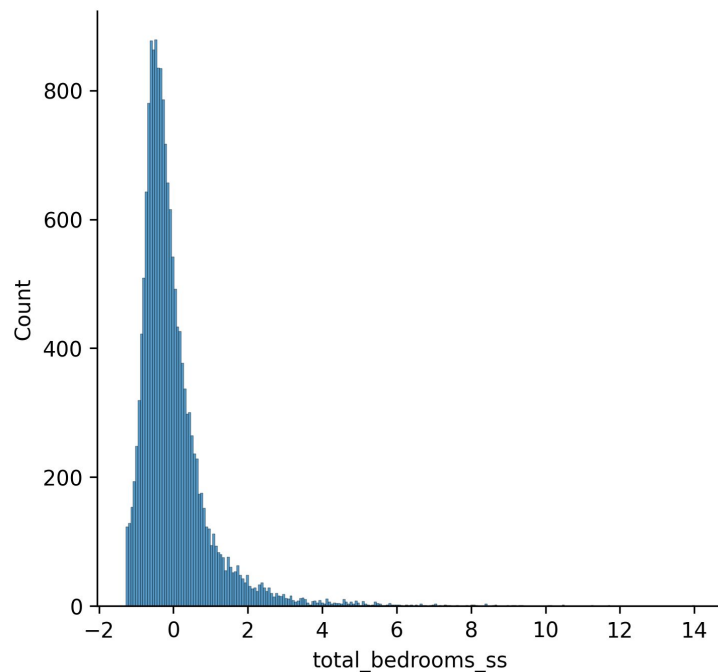
StandardScaler

```
ss = StandardScaler()  
df['total_bedrooms_ss'] =  
ss.fit_transform(np.array(df['total_bedrooms']).reshape(-1, 1))
```

Original



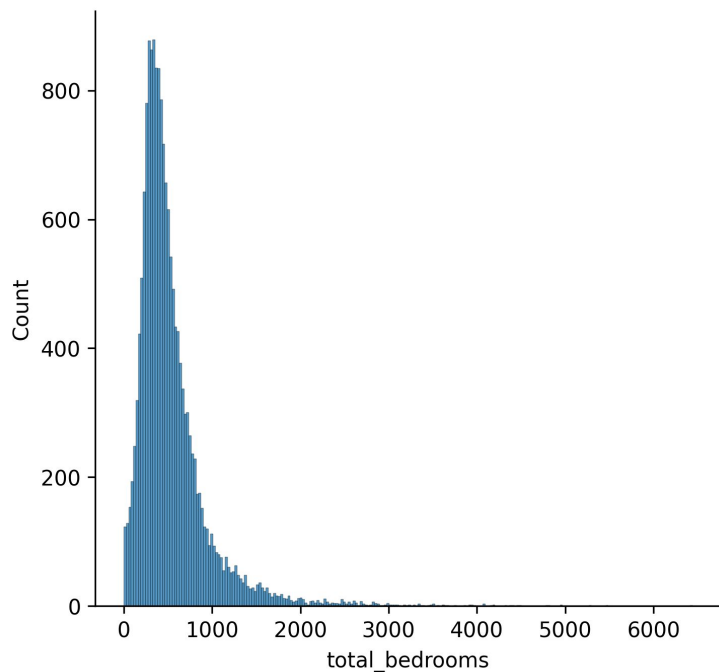
StandardScaler



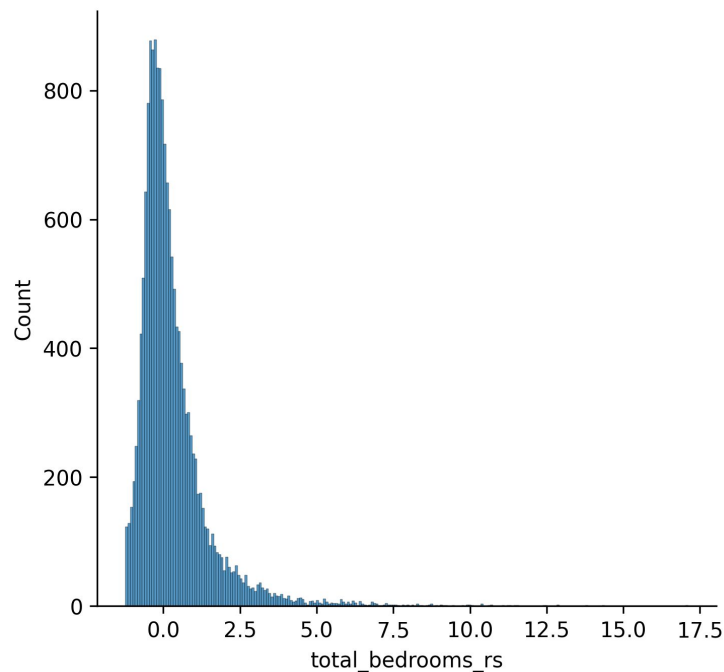
RobustScaler

```
rs = RobustScaler()  
df['total_bedrooms_rs'] =  
rs.fit_transform(np.array(df['total_bedrooms']).reshape(-1, 1))
```

Original



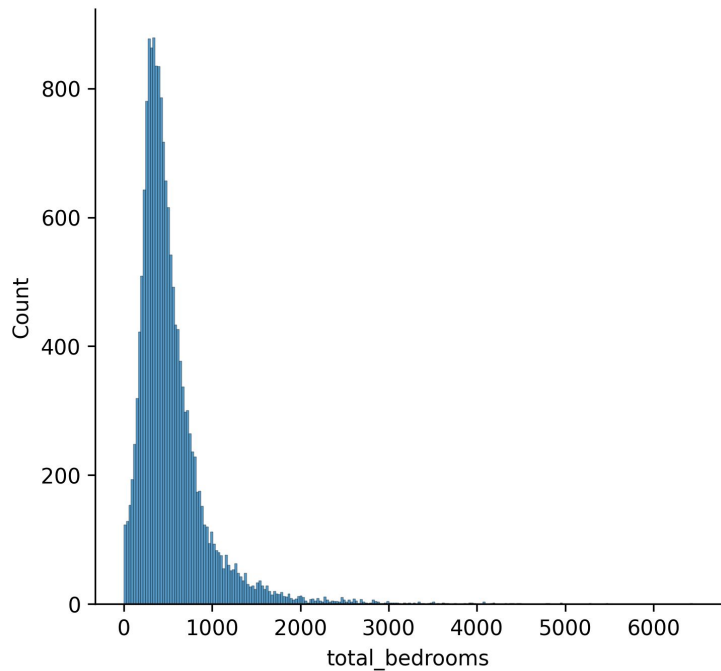
RobustScaler



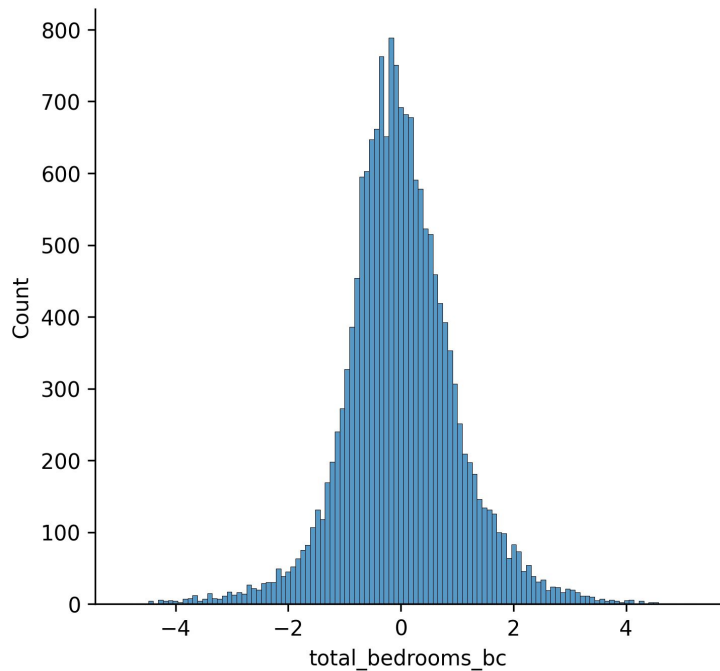
Box-Cox

```
bc = PowerTransformer(method='box-cox')  
df['total_bedrooms_bc'] =  
bc.fit_transform(np.array(df['total_bedrooms']).reshape(-1, 1))
```

Original



Box-Cox



Data Leakage

- Fit preprocessing steps only on training data
 - Apply the learned transformation to validation and test data
 - Never use validation or test statistics during training
- Recommended Workflow
 - Split the data first
 - Fit encoders, imputers, and scalers on the training set
 - Transform validation and test sets using the fitted preprocessing objects

MISSING DATA

Missing data

- Missing values are common in real datasets
- Some models can handle them, but many cannot
- Missingness itself may contain useful information

Why Data May Be Missing

- Users may not provide information
- Some variables may not apply to all instances
- Missing data may be random or systematic
 - Random: values are missing for no clear pattern, such as occasional data entry errors
 - Systematic: values are missing for a specific reason related to the data collection process or the population being studied

1: Drop Rows with Missing Values

- The simplest option is to remove rows that contain missing values
- This may be reasonable when only a small percentage of rows is affected (e.g., $< 5\%$)
- However, dropping rows should not be automatic
 - **Risk:** losing useful data or introducing bias
 - Before dropping rows, check how much data would be lost and whether the missing values are concentrated in specific groups or classes

2: Drop Rows with Too Much Missingness

- Instead of removing every row with at least one missing value, we can use a more selective criterion
 - For example, remove only rows where most columns are missing
- The threshold depends on the dataset and the analysis goal
 - For example: 70%, 75%, or 80% missing values
- The threshold should be chosen after checking how many rows would be removed and how much useful information remains

3: Drop Columns with Many Missing Values

- Consider removing variables with excessive missingness
 - A column with many missing values may be difficult to use reliably
- However, the decision should depend on both the percentage of missing values and the importance of the variable
 - Some variables may still be useful even with substantial missingness
 - Before dropping a column, consider whether imputation or a missing-value indicator would be more appropriate

4: Data imputation

- Imputation replaces missing values with fixed or estimated values
 - This allows us to keep rows and columns that would otherwise be removed
 - Common options include constant values, mean, median, mode, or model-based estimates
- Imputation is useful, but it introduces assumptions about the data
 - The chosen strategy should match the variable type, distribution, and modeling goal
 - Again, always fit imputation using training data only

Statistical Imputation

- **Mean:** useful for roughly symmetric numerical data
- **Median:** safer when outliers are present
- **Mode:** common for categorical variables

Predictive Imputation

- If the dataset is not too large, another idea is to use ML models to 'predict' the missing values.
- The column with missing values is assigned as the target variable and the remainder are used as predictors
- Well-known options:
 - KNN imputation
 - Random Forest imputation (MissForest)

Missing Data: Key Takeaways

- Small amounts of missing data may be dropped
- Columns with too much missingness may be removed
- Data imputation via:
 - Generic values
 - Basic statistics
 - ML models
- Imputation is common, but not always harmless
 - Missingness should be analyzed before choosing a strategy

Practice

- Handling missing data with pandas and scikit-learn
 - Load the titanic dataset
 - `df =`
`pd.read_csv('https://raysonlaroca.github.io/teaching/data-science/datasets/titanic.csv')`
 - Inspect missing values by row and by column
 - Determine appropriate handling strategies for each variable:
 - Drop, impute, or flag missing values?
 - Use mean, median, mode, or model-based imputation?

Fixed Imputation

```
1 # Replace missing values in the 'Cabin' column
2 # with a fixed value ('UNK')
3
4 # Option 1: Using replace directly
5 df['Cabin'] = df['Cabin'].replace(np.nan, 'UNK')
6
7 # Option 2: Using fillna with inplace=True
8 df.fillna(value={'Cabin': 'UNK'}, inplace=True)
```

Statistical Imputation

```
1 # Regardless of the chosen statistic (mean, mode, median),
2 # the simplest way is to use the fillna() function
3
4 # Using fillna()
5 df.fillna(value={'Age': df['Age'].median()}, inplace=True)
6
7 # Another way to achieve the same result (loc[]):
8 df.loc[df['Age'].isna(), 'Age'] = df['Age'].median()
9
10 # Note: Using .loc provides more control and flexibility.
11 # For example, it allows you to apply conditions to specific rows or columns,
12 # which can be useful in more complex imputation scenarios.
```

Project

- Use the remainder of today's class to work on your project
 - Review your dataset and identify which variables need preprocessing
 - Decide how to handle missing data
 - Choose how to encode categorical variables
 - Check whether numerical features need scaling, standardization, or another transformation
 - Make sure preprocessing choices are justified and consistent with the models you plan to use
 - Be careful to avoid data leakage: fit preprocessing steps only on the training data